

Sisteme și algoritmi distribuiți

Curs 2

Cuprins

- Modele de comunicație
- Comunicație prin mesaje
- Mesaje persistente
- Ceasuri

Comunicație

În limbajul natural (uman) exprimarea unui mesaj cuprinde:

- Dialect
- Limbă
- Cuvânt
- Alfabet

Comunicație

Premise necesare pentru comunicație:

- Sursa și destinația sunt conectate fizic
- Sursa alege alfabetul, limba etc., în care codifică mesajul
- Destinația decodifică mesajul (recunoaște alegerile sursei)

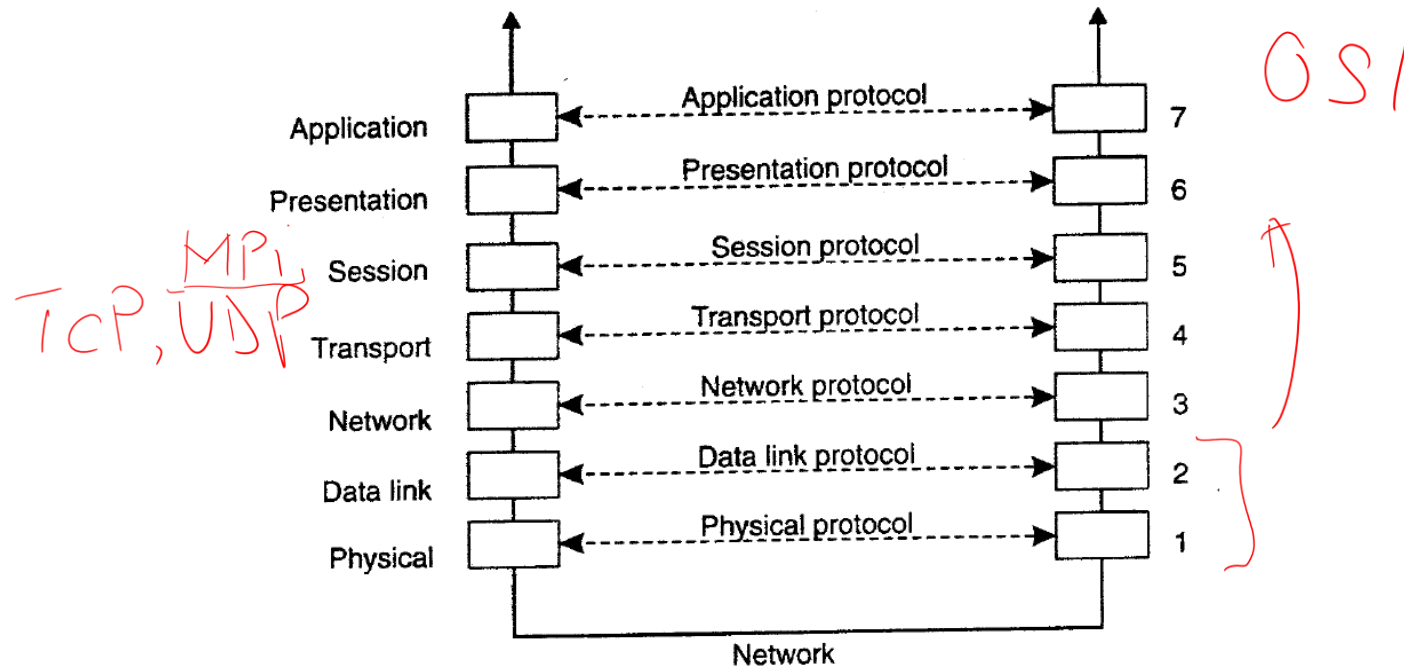
Modele de comunicație

A comunică cu B:

- A produce un mesaj
- Apelează o primitivă pentru a trimite mesajul *send*
- B apelează o primitivă pentru a primi mesajul *recv*

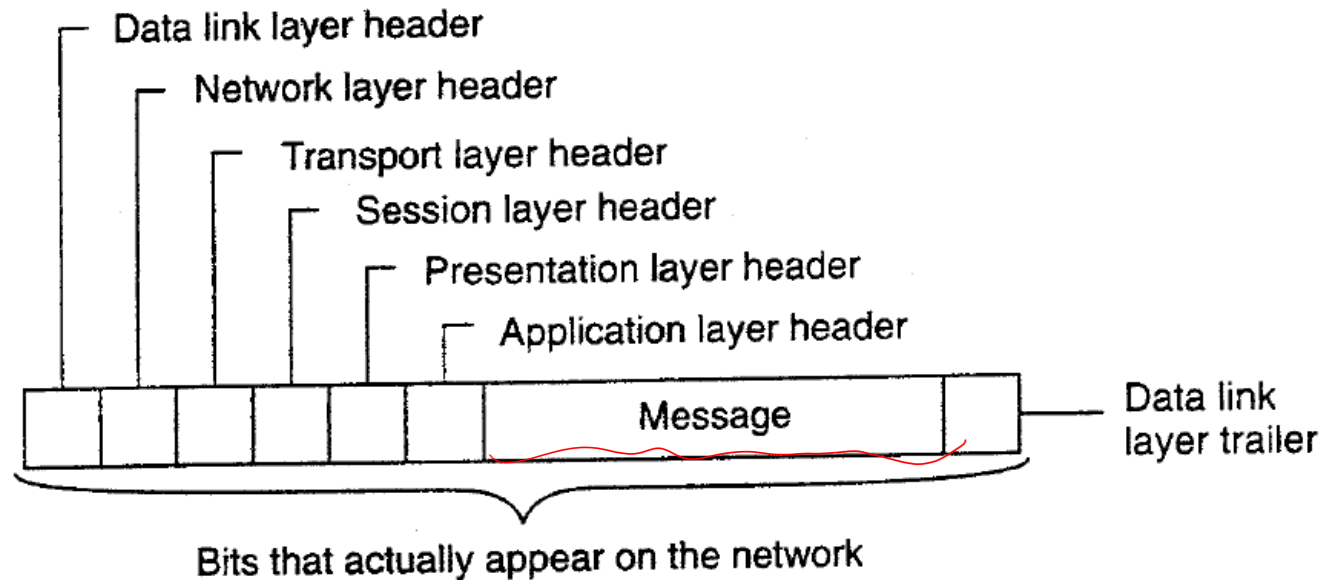
Protocoloale de comunicație

Protocol = algoritm care specifică formatul, conținutul și sensul mesajelor trimise și primite de către un proces.



Protocoloale de comunicație

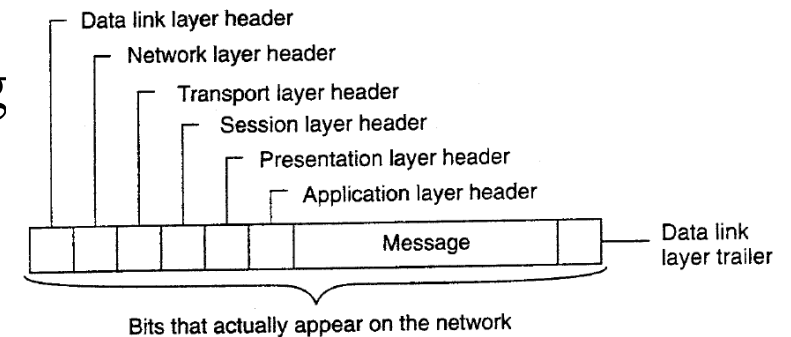
Protocol = algoritm care specifică formatul, conținutul și sensul mesajelor trimise și primite de către un proces.



Ex.: User Datagram Protocol (UDP)

Protocol de comunicație la nivelul (layer) de transport

- Fără conexiune = sursa comunică pachete către destinație fără a negocia o conexiune (implicit fără gestiunea pachetelor)
- Lightweight = nu necesită ordonare sau istoric de pachete; comunicația mesajului începe de la primul pachet
- Singurul mecanism de siguranță este *checksum*, nu există verificarea succesului transmisiei.
- Folosit în aplicațiile de streaming, broadcasting



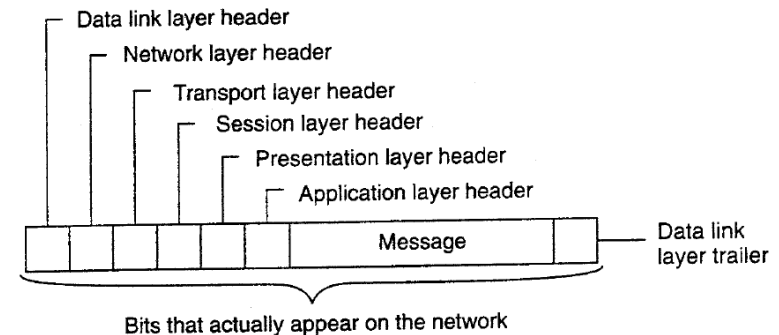
Ex.: User Datagram Protocol (UDP)

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Length																Checksum															
8	64	Data																															
12	96																																
⋮	⋮																																

Ex.: Transmission Control Protocol (TCP)

Protocol de comunicație la nivelul (layer) de transport

- Orientat pe conexiune = înainte de interschimbarea de mesaje dintre sursă și destinație, se realizează o conexiune.
- Heavyweight = primele 3 pachete necesare pentru conexiune prin socket.
- Mecanisme de ACK, retransmisie și time-out
- Cuprinde trei etape:
 - Stabilire conexiune
 - Transfer date
 - Oprire conexiune
- Divide mesajul în porțiuni și creează segmente de dimensiuni fixe

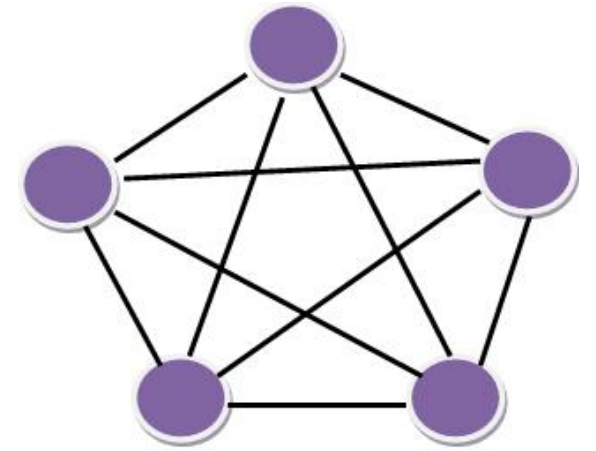
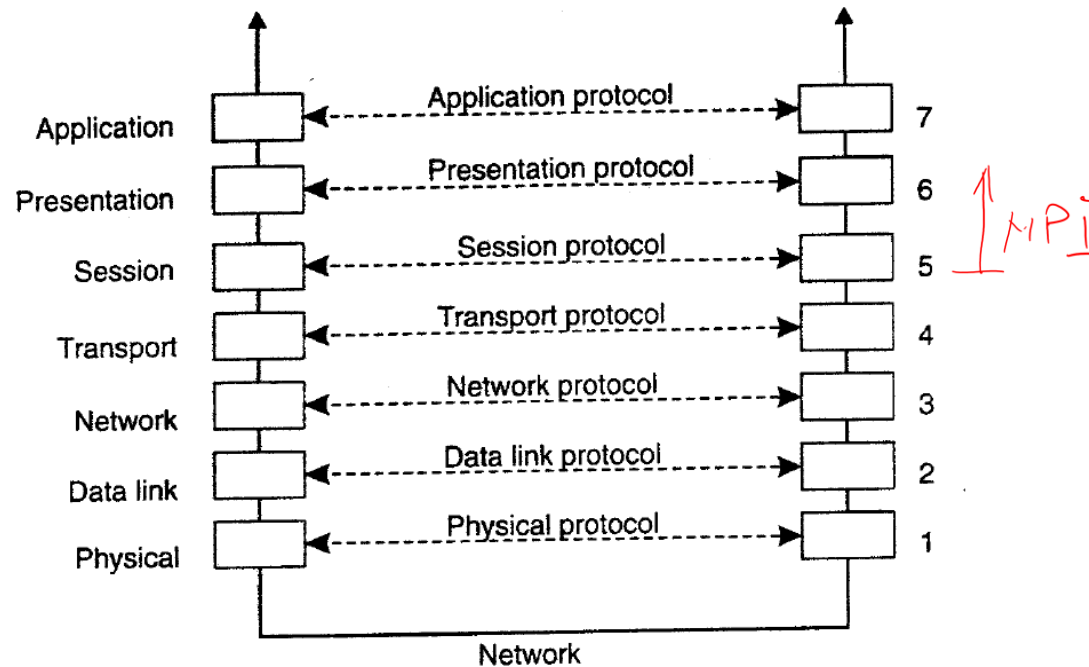


Ex.: Transmission Control Protocol (TCP)

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Sequence Number																															
8	64	Acknowledgement Number (meaningful when ACK bit set)																															
12	96	Data Offset				Reserved				C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window															
16	128	Checksum																Urgent Pointer (meaningful when URG bit set) ^[18]															
20	160	[Options] If present, Data Offset will be greater than 5. Padded with zeroes to a multiple of 32 bits, since Data Offset counts words of 4 octets.																															
:	:																																
56	448																																
60	480	Data																															
64	512																																
:	:																																

Modele de comunicație

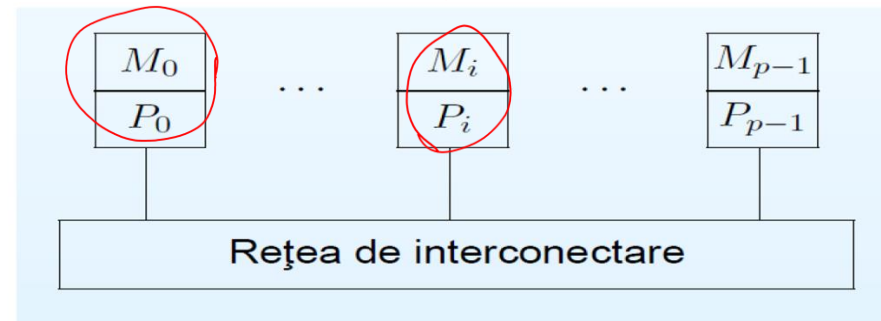
1. *Comunicație prin mesaje*
2. Comunicație prin mesaje persistente
3. Comunicație colectivă



<http://4.bp.blogspot.com/-LLvGwI5e-4UMadzP7U/AAAAAAAAAHY/2AxHYokPHzk/s1600/all+channel.jpg>

Comunicația prin mesaje: Modelul SPMD

- MIMD cu memorie distribuita
- Paradigma SPMD (Single Program Multiple Data): toate procesoarele execută același program, dar fiecare utilizează un set propriu de date.
- În general, execuția programului nu este sincronă;
- Deși toate procesoarele văd același program, instrucțiunile executate nu sunt identice



Modelul SPMD

- Procesoarele sunt numerotate $\underline{0}, \dots, p-1$
- Numerotarea nu este statică, ci se realizează la momentul execuției.
- Există primitive care returnează adresa unui procesor (e.g. MPI_rank) MPI
- Procesoarele pot executa instrucțiuni diferite în funcție de adresa lor

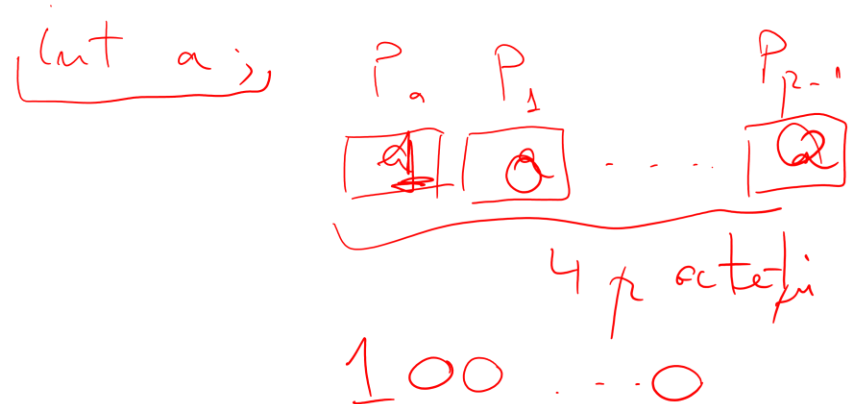
1. rank $\leftarrow$ adresa proprie = MPI_rank() $\Rightarrow$ $\frac{\text{Rank} \leftarrow 0}{\text{Rank} \leftarrow 1}$

2. dacă rank = 0 atunci

1. a $\leftarrow$ 1

3. altfel

1. a $\leftarrow$ 0

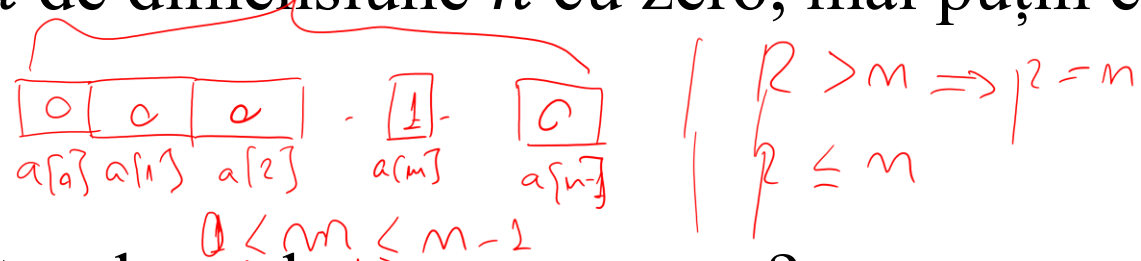


Modelul SPMD - variabile

- O variabilă a unui program SPMD este multiplicată (de p ori) : fiecare procesor deține un exemplar, asupra căruia are control complet.
- Un procesor nu poate modifica variabile din memoria altui procesor.
- Putem interpreta variabila a ca un vector cu p elemente: fiecare procesor P_i deține componenta i a vectorului. Cu toate acestea i reprezintă un index global al datelor din a .
- Programul inițializează a cu 0 pe toate componentele, cu excepția primei componente (care este 1).

Modelul SPMD - problemă

Inițializați un vector a de dimensiune n cu zero, mai puțin elementul a_m , care să fie 1.



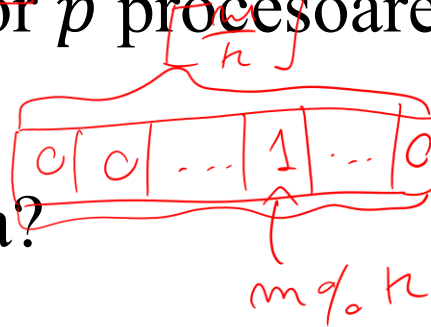
- Cum distribuim vectorul a celor p procesoare?

$$k = \left\lfloor \frac{n}{p} \right\rfloor$$

$$P_1: k \cdot 0 : k(1+1) - 1$$

$$P_{p-1}: k(p-1) : kn$$

- Cum facem efectiv inițializarea?



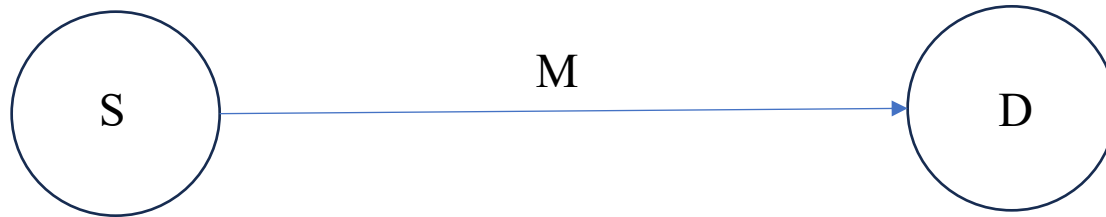
int rank ← adresa P_i

dacă $\text{rank} = \left\lfloor \frac{n}{k} \right\rfloor$

$$a[m \% k] = 1;$$

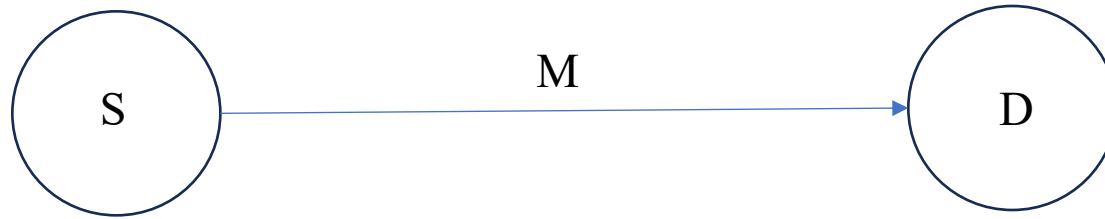
Modelul de comunicație prin mesaje

- Singura modalitate de comunicare între procesoare este transmiterea de mesaje
- **Operația de bază:**



Modelul de comunicație prin mesaje

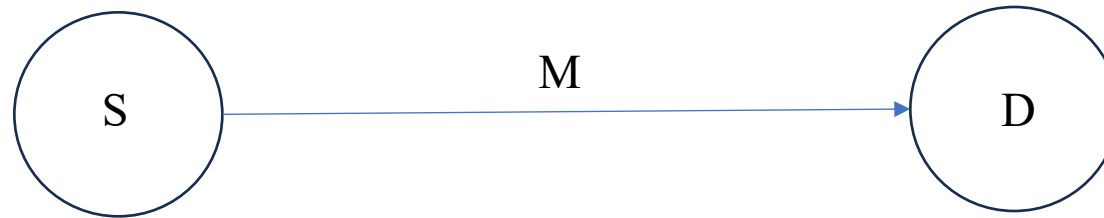
- Operația de bază:



- Procesorul sursă transmite prin rutina **send**
- Procesorul destinație recepționează prin rutina **recv**
- Sintaxă generală:
 - `send(date, dest)` *pointer data*
 - `recv(date, sursa)` *pointer
zonă liberă*
- date = locație (**buffer**) din care se preiau/depun mesajele transmise
- sursa/dest = adresa procesorului cu care se comunică

MP - corectitudine

- Operația de bază:



- Orice operație de **send** trebuie însoțită de una de **recv**
- Per ansamblu, vom avea perechi send-recv

Exemplu: Procesorul i transmite un mesaj M vecinilor de la stânga, respectiv dreapta (pe o topologie inel)

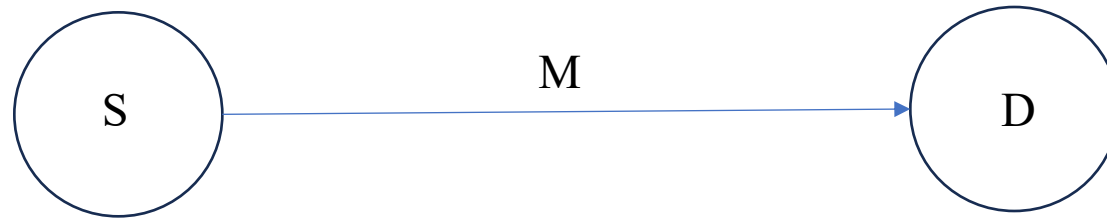
1. **dacă rank = i atunci**

1. send(M , $(i + 1) \bmod p$);
2. send(M , $(i - 1) \bmod p$);

2. Altfel **dacă rank = $(i + 1) \bmod p$ atunci** recv(M , i);
3. Altfel **dacă rank = $(i - 1) \bmod p$ atunci** recv(M , i);

MP - sincronizare

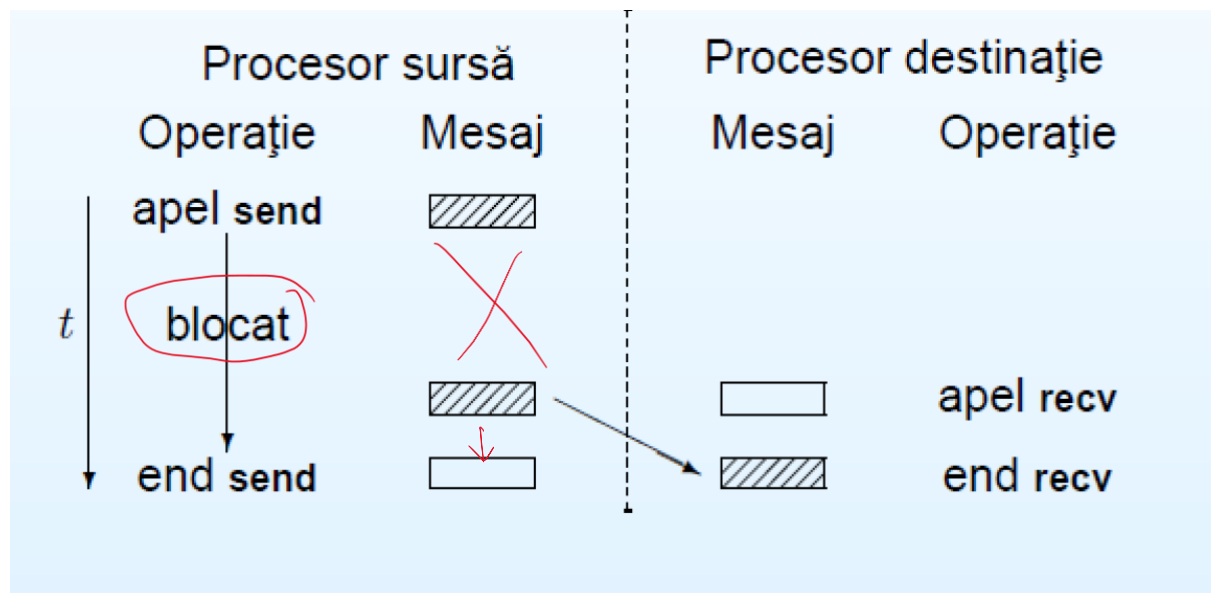
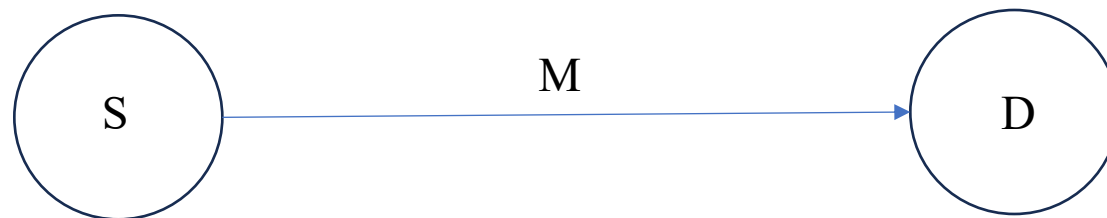
- Operația de bază:



- Momentul apelului primitivei **send** este în general diferit față de momentul apelului **recv**
- Ce se întâmplă din momentul apelului primei primitive până la finalizarea comunicației?
 - Răspunsuri posibile: (i) așteaptă (**comunicație blocantă**); (ii) poate executa alte operații (**comunicație non-blocantă**)

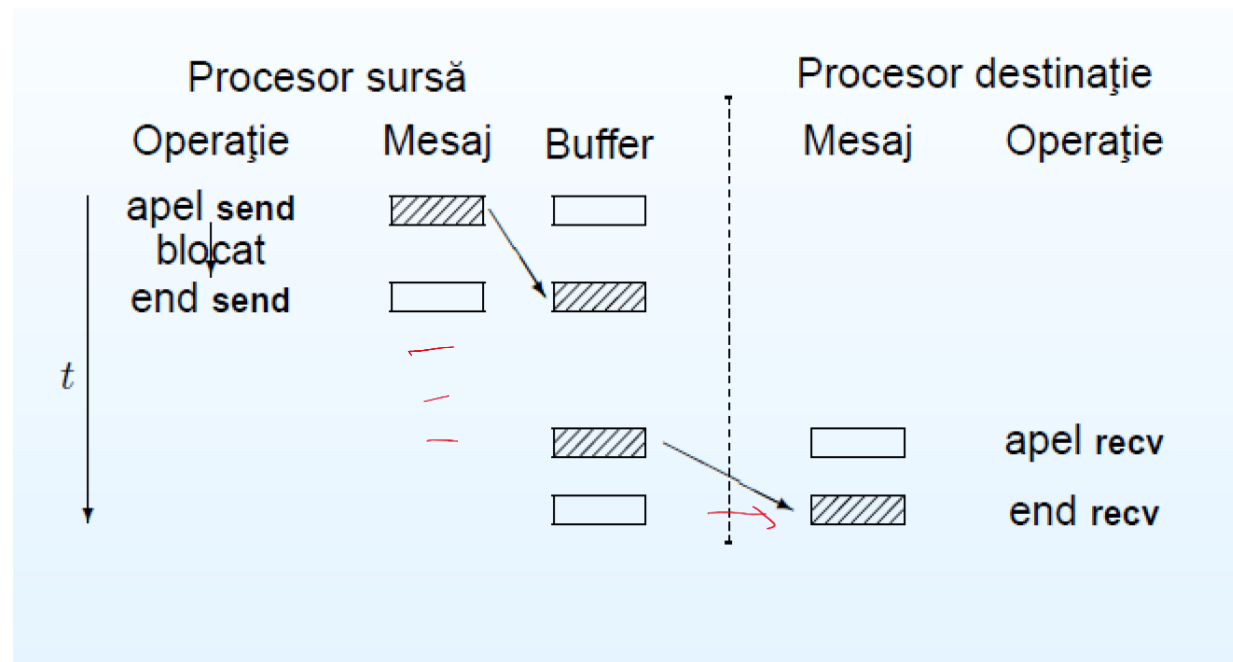
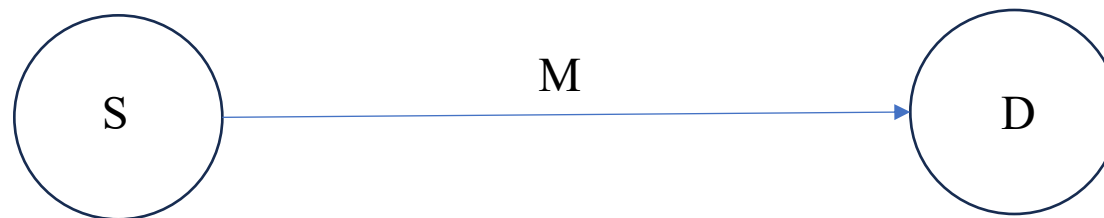
MP – comunicație blocantă

- Operația de bază:



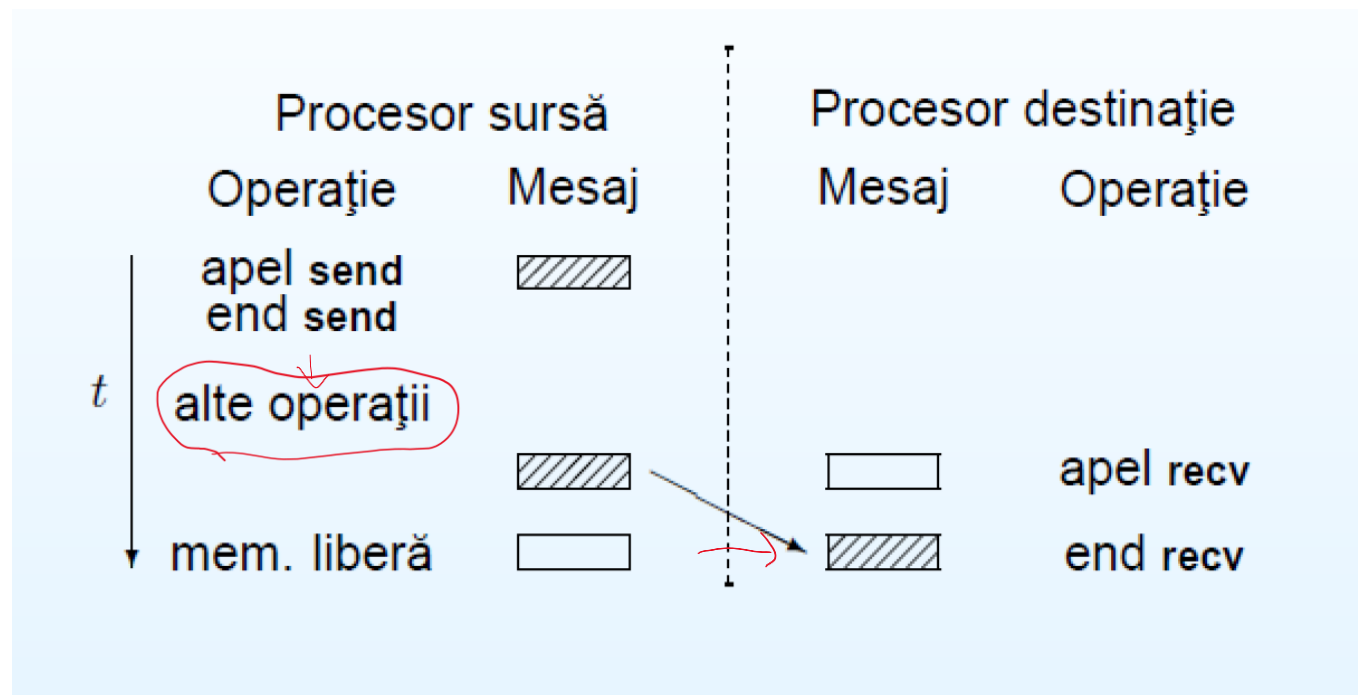
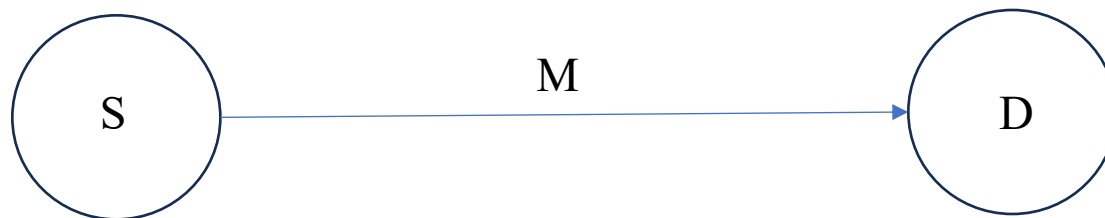
MP – comunicație blocantă prin buffer

- Operația de bază:



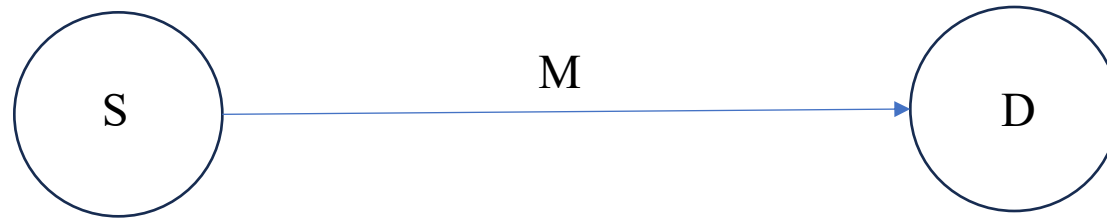
MP – comunicație non-blocantă

- Operația de bază:



MP – comunicație non-blocantă

- **Operația de bază:**



Alte primitive:

- Așteaptă (**Wait**) – așteptarea terminării comunicației

Utilizare:

1. **send**(*date*, dest);
2. execută operații care nu modifică date;
3. așteaptă terminarea **send**;
4. modifică date;

MP – comparație

- Comunicația non-blocantă asigură o ocupare mai bună a procesoarelor
- Erorile de programare:
 - Comm blocantă: blocarea execuției
 - send(m1, (rank + 1) mod p) → tag = 1
 - recv(m2, (rank - 1) mod p) ← tag = 1
 - Comm non-blocantă: alterarea zonei de memorie după apelul **send**

Standardul MPI (Message-Passing Interface)

- Standard care descrie primitivele de comunicație în paradigma SPMD
- Implementări: OpenMPI, mpich2 etc.
- O rutină MPI se execută în cadrul unui grup de procesoare (communicator)
- Într-un communicator procesoarele sunt numerotate $\{0, \dots, p - 1\}$

C:

int MPI_Comm_rank(MPI_Comm com, int * my_rank)

int MPI_Comm_size(MPI_Comm com, int *p)

mpich 2

Python:

int comm.Get_rank()

int comm.Get_size()

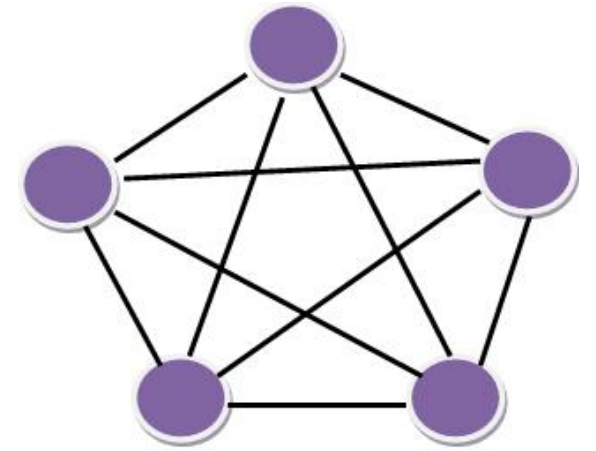
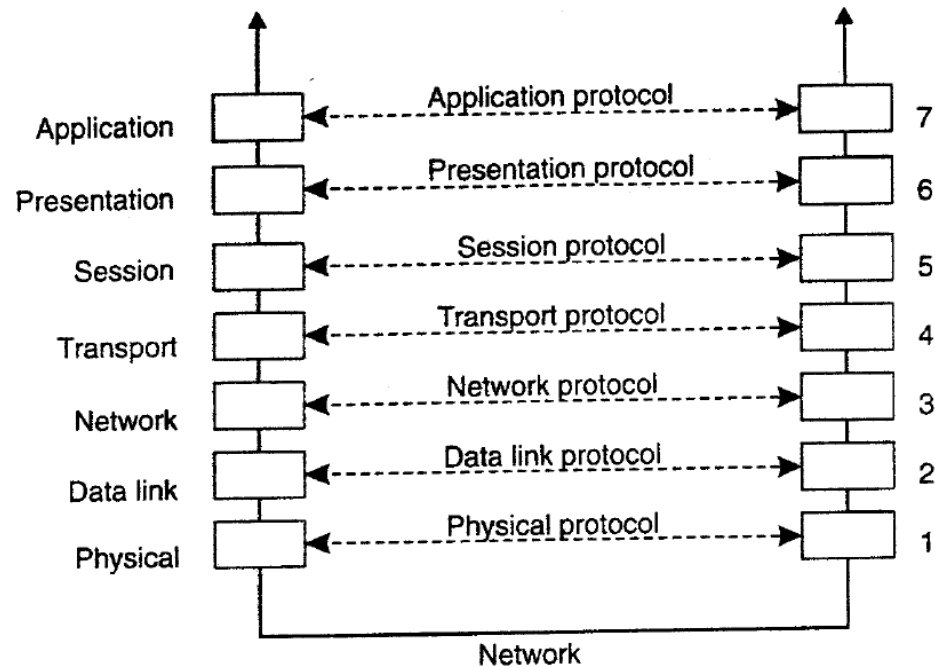
mpir4py

MPI primitives

Primitive	Meaning
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until receipt starts
MPI_sendrecv	Send a message and wait for reply
MPI_isend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

Modele de comunicație

1. Comunicație prin mesaje
2. *Comunicație prin mesaje persistente*
3. Comunicație colectivă



<http://4.bp.blogspot.com/-LLvGwI5e-4UMadzP7U/AAAAAAAAAHY/2AxHYokPHzk/s1600/all+channel.jpg>

Comunicație prin mesaje persistente (MQS)

Message Queuing Systems = sistem în care aplicațiile comunică prin inserarea de mesaje în cozi specifice;
“O abstractizare a căsuței poștale”

- Sursa (producer) are garanția că mesajul său va fi eventual inserat în coada destinatarului (receiver)
- Nu avem garanții despre momentul când (sau dacă) mesajul va fi citit
- Sursa și destinatarul execută complet independent unul de celălalt.

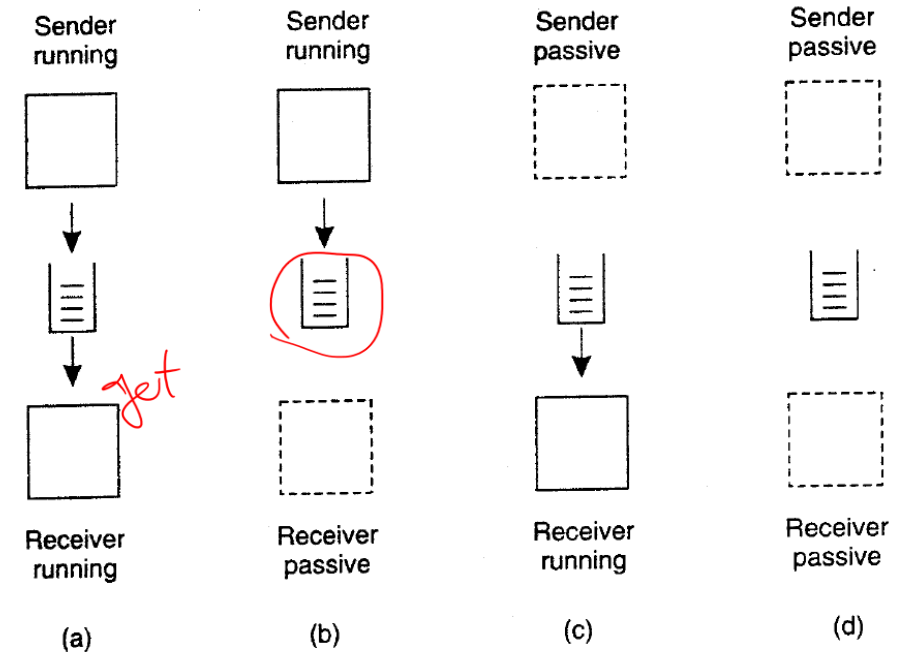
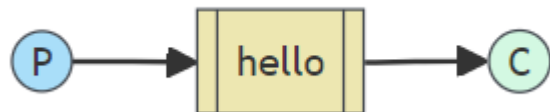


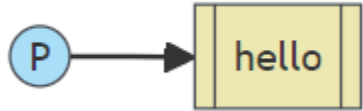
Figure 4-17. Four combinations for loosely-coupled communications using queues.

MQS primitives

Primitive	Meaning
<u>Put</u>	Append a message to a specified queue
<u>Get</u>	Block until the specified queue is nonempty, and remove the first message
Poll	Check a specified queue for messages, and remove the first. Never block
Notify	Install a handler to be called when a message is put into the specified queue

Figure 4-18. Basic interface to a queue in a message-queuing system.

Rabbit MQ example - sender



```
#!/usr/bin/env python
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
```

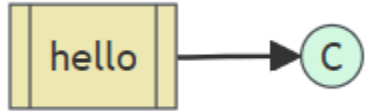
- Client RabbitMQ ⇒ modul Python pika
- Asigurăm existența cozii în care scriem mesajele (cu numele 'hello')

```
channel.queue_declare(queue='hello')
```

```
channel.basic_publish(exchange='',
                      routing_key='hello',
                      body='Hello World!')
print(" [x] Sent 'Hello World!'" )
```

```
connection.close()
```

Rabbit MQ example - receiver



```
#!/usr/bin/env python
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
```

- Se realizează conexiunea identică
- Dacă nu cunoaștem a priori existența cozii create de sursă, asigurăm existența cozii în același fel

```
channel.queue_declare(queue='hello')
```

```
def callback(ch, method, properties, body):
    print(f" [x] Received {body}")
```

```
channel.basic_consume(queue='hello',
                      auto_ack=True,
                      on_message_callback=callback)

print(' [*] Waiting for messages. To exit press CTRL+C')
channel.start_consuming()
```

```
if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
        print('Interrupted')
    try:
        sys.exit(0)
    except SystemExit:
        os._exit(0)
```


Rabbit MQ example

Execuție pe receiver:

```
python receive.py  
# => [*] Waiting for messages. To exit press CTRL+C
```

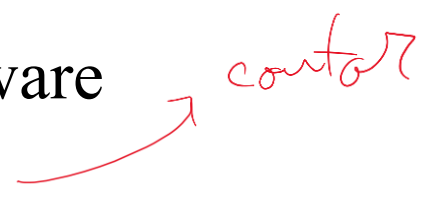
```
# => [*] Waiting for messages. To exit press CTRL+C  
# => [x] Received 'Hello World!'
```

Execuție pe sender:

```
python send.py  
# => [x] Sent 'Hello World!'
```

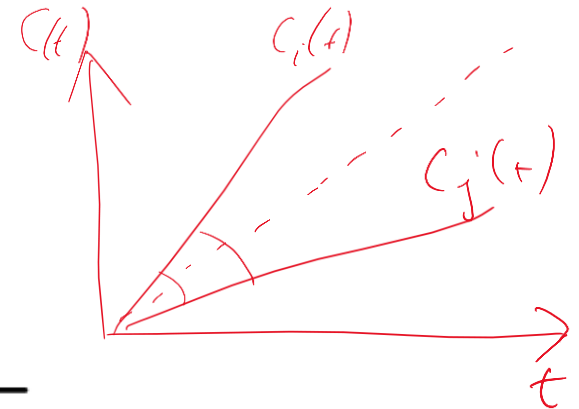
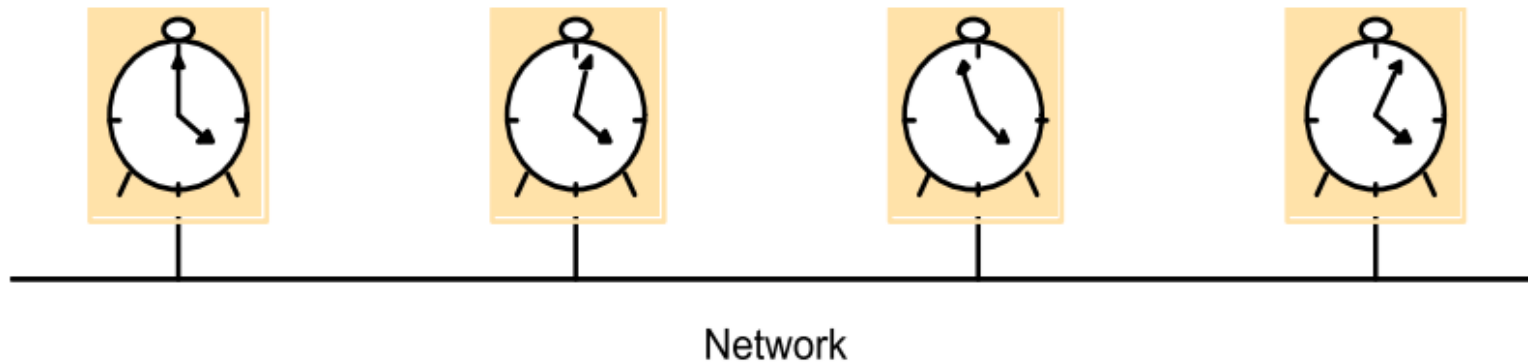
Ceasuri

Timp

- Ceasurile fizice în computere sunt realizate prin contorizare
 - Ceasuri atomice: drift 1s/150 milioane de ani
 - Ceasuri de sistem
 - Ceasuri de timp real: alimentate prin baterie (funcționează chiar dacă sistemul este oprit)
- $h(t)$ rata (viteza) ceasului hardware
- $\underline{H(t)} = \int_0^t h(\tau) \tau$ valoarea ceasului hardware
- Ceas logic (registru): $\underline{C(t)} = \underline{\alpha H(t)} + \beta$ 
- $C(t)$ este un scalar crescător și se actualizează prin citiri ale lui $H(t)$

Timp

- In SD, ceasurile locale sunt **decalate** și **întârziate**
- Decalaj între nodurile (i, j) la momentul t : $|C_i(t) - C_j(t)|$
- Întârziere între nodurile (i, j) la momentul t : $|\frac{d}{dt}C_i(t) - \frac{d}{dt}C_j(t)|$

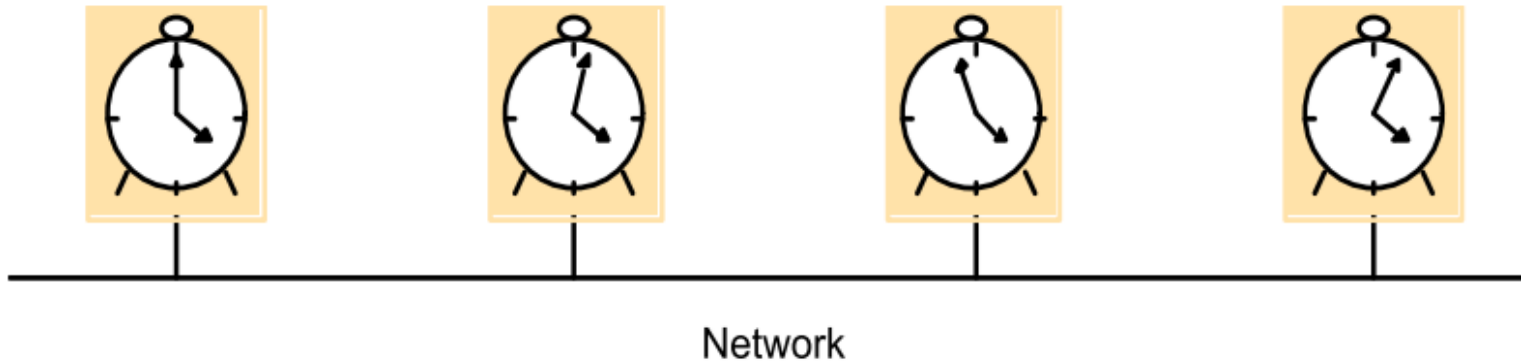


Problema sincronizării

Folosind referințe externe sau interne, problema sincronizării ceasurilor se reduce la asigurarea relației (de precizie):

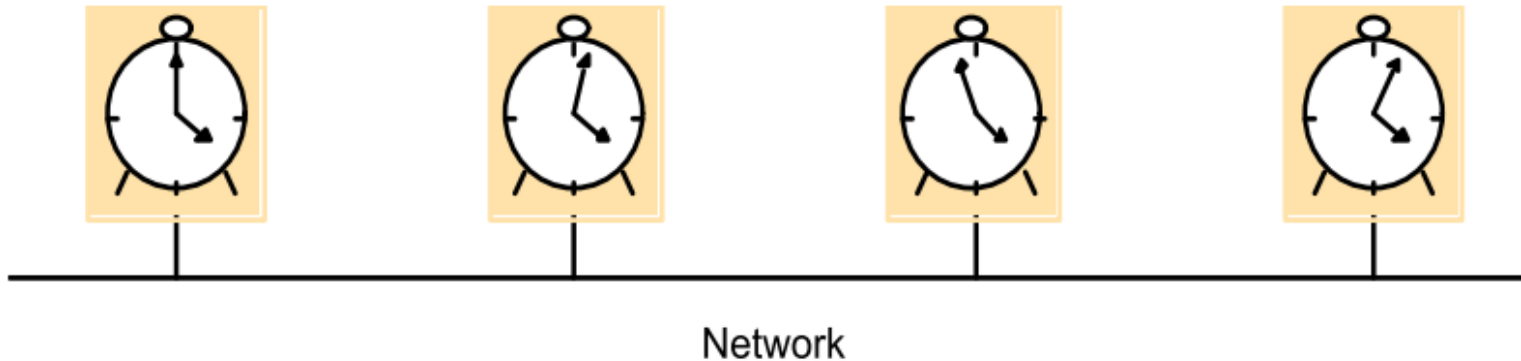
$$|C_i(t) - C_j(t)| \leq \rho \quad \forall t \geq t_0, \forall i, j.$$

- Proprietate globală atinsă prin operații la nivel local!



Problema sincronizării

- Sincronizare externă: referința este un ceas absolut extern e.g. Coordinate Universal Time (UTC), GPS etc.
 - Algoritmul lui Cristian
- Sincronizare internă: referința este o valoare stabilită în rețea
 - Algoritmul de Medie (Berkeley Algorithm)



UTC

UTC este standardul primar de timp în lume, transmis prin: radio, linii telefonice line, satelit (GPS) etc.

Protocolele populare de sincronizare folosesc referința UTC și urmăresc să asigure relația (acuratețe):

$$|\underline{S(t)} - C_i(t)| < \underline{\rho} \quad \forall i, \underline{\forall t}$$

Dacă asigurăm acuratețea ρ atunci ce precizie garantăm?

$$\left. \begin{array}{l} |S(t) - C_i(t)| < \rho \\ |S(t) - C_j(t)| < \rho \end{array} \right\} \Rightarrow |C_i - C_j| \leq |S - C_i| + |S - C_j| < 2\rho$$

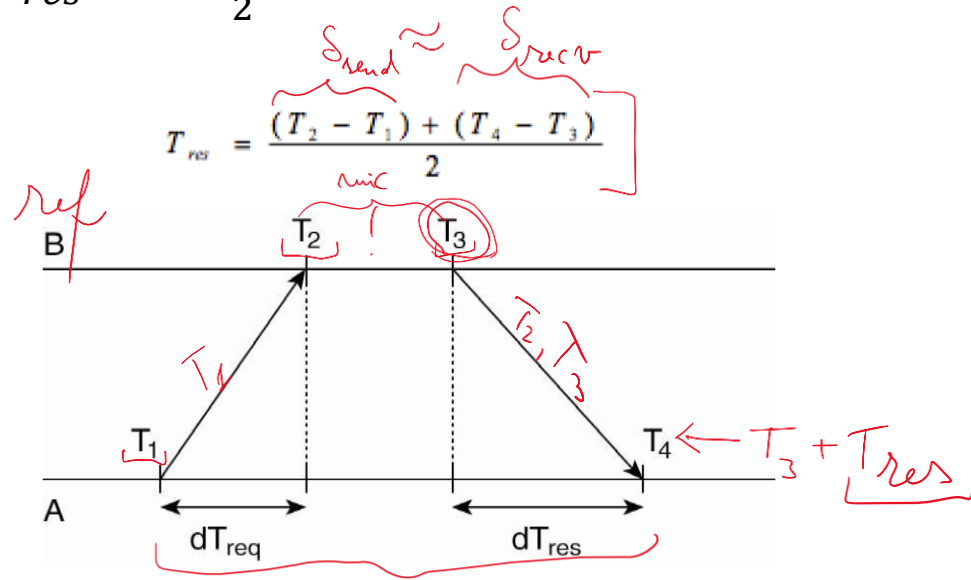
Algoritmul lui Cristian

- Ipoteza 1: Întârzieri pe comunicație simetrice și mărginite (rețele LAN)
- Ipoteza 2: Există un nod de referință (pasiv) R cu unicul rol de a furniza referința
- Nodurile care nu sunt referința se vor sincroniza cu ceasul referinței

Algoritmul lui Cristian

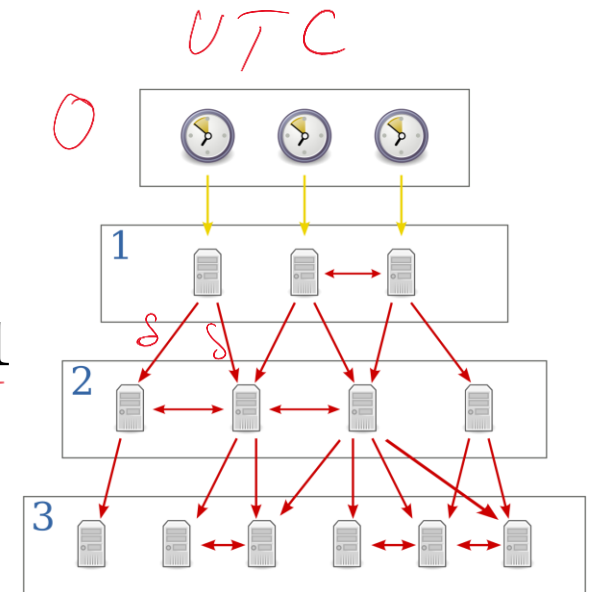
Procedura AC: Nodul P cere periodic valoarea timpului de la R:

- T_1 : send request; T_4 : primește reply
- P primește val. T_2 și T_3 de la R, ajustează $C(t) = T_3 + T_{res}$ (T_{res} timp livrare mesaj)
- Folosește estimarea $T_{res} \approx \frac{T_{req} + T_{res}}{2}$



Network Time Protocol (NTP)

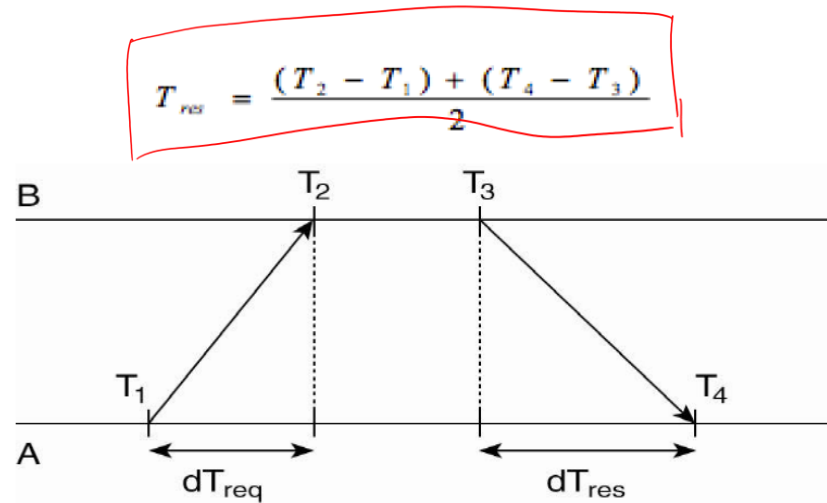
- Implementare standard în rețele a alg. lui Cristian
- Peste multiple măsurători, calculăm T_{res} minim
- Ierarhie a serverelor de timp:
 - un TS de nivel k se sincronizează după unul de nivel $\leq$ $k - 1$
 - Nivelul 0 este referința UTC



Exemplu

La 5:08: ^{T₁}15.100, P trimite mesaj de request către R. La 5:08: ^{T₄}15.900, P primește răspuns de la B cu valoarea 5:08:25.300 (T₂ = T₃)

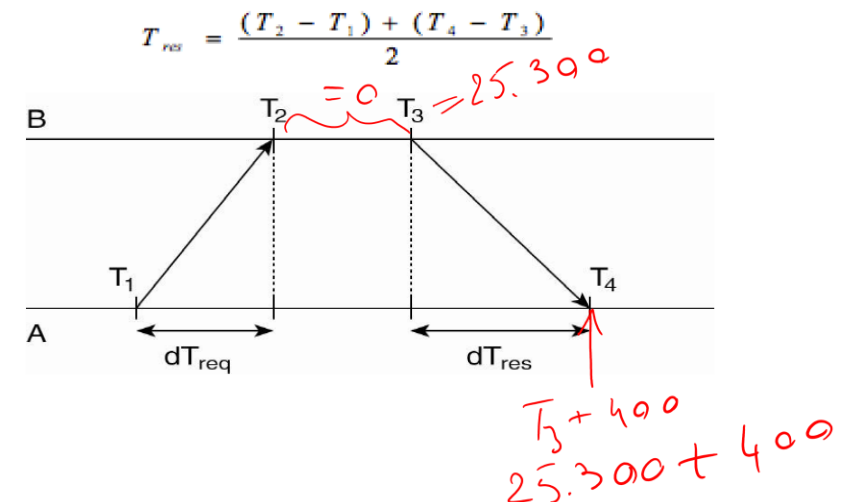
- Care este valoarea ajustată a ceasului local ^{T₂ = T₃}T₄ = C_P(t) după sincronizare?



Exemplu

La 5:08:15.100, P trimite mesaj de request către R. La 5:08:15.900, P primește răspuns de la B cu valoarea 5:08:25.300 ($T_2 = T_3$)

- Send req la $T_1 = 5:08:15.100$
- Primește răsp. la $T_4 = 5:08:15.900$
- Mesajul este $T_3 = T_2 = C_R(t) = 5:08:25.300$
- Durata totală $T_4 - T_1 = 800\text{ ms}$ / 2 } 0
- Estimare: mesaj generat în urmă cu 400 ms } T_{res}
- Set $C_P(t) = T_{serv} + 400 = 5:08.25.700$

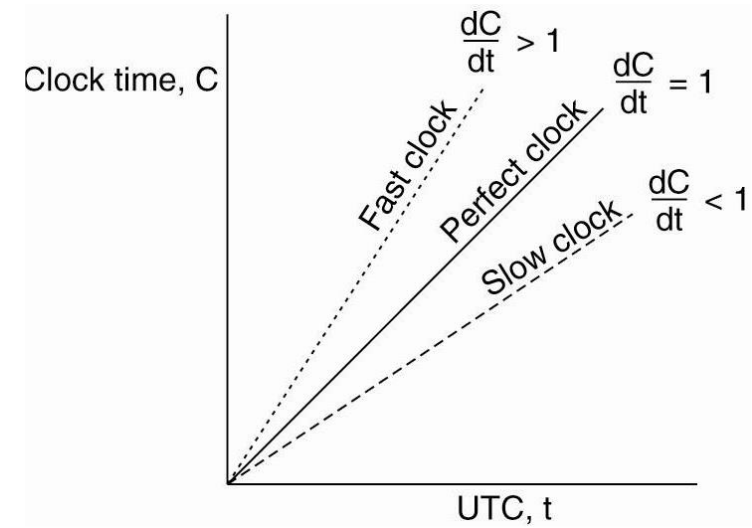


Sincronizare internă

Sincronizarea internă a ceasurilor locale în DS presupune (precizie)

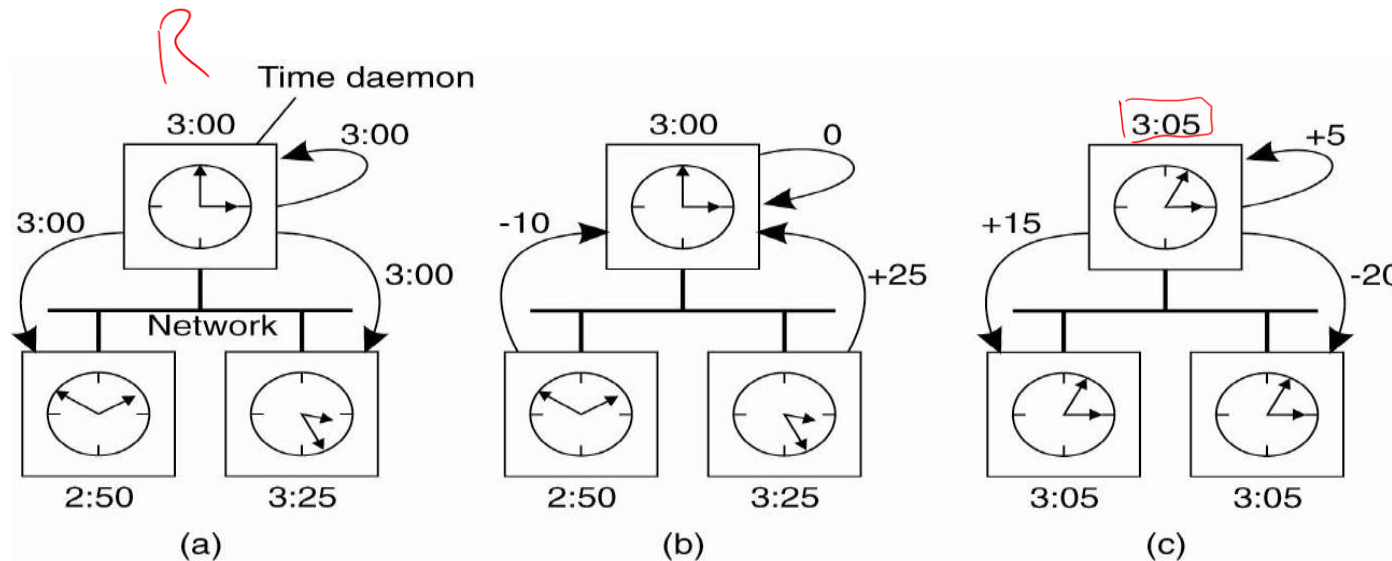
$$|C_j(t) - C_i(t)| < \rho \quad \forall i, j, t$$

- Necesită algoritmi complet distribuiți
- Problema este una de consens *dinamic* distribuit
- Vom reveni la ea în cursurile următoare



Algoritmul de medie (Berkeley Algorithm)

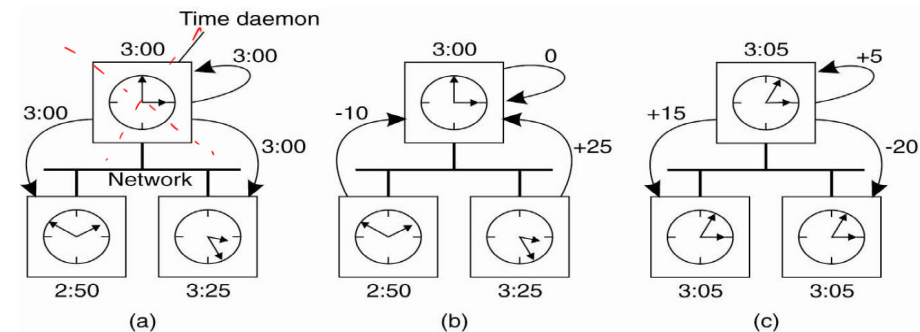
- Referința este unul din nodurile rețelei, ales eventual prin proceduri de leader-election
- Restul nodurilor urmăresc alinierea ceasurilor cu referința (consens)



Algoritmul de medie (Berkeley Algorithm)

- Referința este unul din nodurile rețelei, ales eventual prin proceduri de leader-election
- Restul nodurilor urmăresc alinierea ceasurilor cu referința (consens)
- Pe scurt: la iterația t

- best* [• R difuzează valoarea $C_R(t)$
- [• P_i calculează întârzierea locală $\delta_i = |C_R(t) - C_i(t)|$ și răspunde lui R
- best* [• R distribuie ajustările pentru $C_i(t)$



Alternative

1. If two machines don't interact, there is no need to synchronize them (Leslie Lamport)

2. Dinamica întârzierii per link $\frac{T_{req} + T_{res}}{2}$ depinde de mai mulți factori

3. Adesea, contează ca procesele să convină asupra **ordinii** evenimentelor, și nu asupra timpului la care au avut loc (vezi Ceasuri Logice).

References

- [1] Seif Haridi, <https://canvas.instructure.com/courses/902299/modules>
- [2] A.S. Tanenbaum, M.V. Steen, *DISTRIBUTED SYSTEMS: Principles and Paradigms*, Pearson Prentice Hall, Second Edition, 2007.
- [3] A. D. Kshemkalyani and M. Singhal, *Distributed Computing: Principles, Algorithms, and Systems*, Cambridge University Press, 2008.
- [4] G. Tel, *Introduction to Distributed Algorithms*, Cambridge University Press, 2000.
- [5] M. Raynal, *Distributed Algorithms for Message-Passing Systems*, Springer-Verlag, 2013.